



Take-home Assignment — Cleaning Robot API

Purpose

Build a small REST API that remotely controls a household cleaning robot.

The purpose of this assignment is to test your familiarity with best practices in software design and development. The API contract below is intentionally precise so that submissions can be tested consistently.

Develop the solution as you would deliver a professional application:

- simple solutions are welcome, but avoid quick-and-dirty implementations;
- follow the design and development practices you would defend in a code review;
- the final solution should be easy for someone else to read, understand, and extend.

Expected time: 4–5 hours, depending on your familiarity with FastAPI and Pydantic. This range is a scope signal, not a hard deadline: it tells you how much to build, not how long to work. A focused, well-structured solution within this scope is preferable to a more feature-complete one that goes well beyond it. If you go over, prioritize the *Evaluation priorities* below over finishing every detail.

AI assistance

AI coding assistants are explicitly permitted, but completely optional. This assignment is self-contained and will be evaluated with the same criteria whether you use a paid tool, a free tool, or no AI assistant.

Technical requirements

- Use Python 3.11 or newer and FastAPI.
- Choose any additional mainstream libraries you consider appropriate.
- Keep cleaning-session history for the lifetime of the running application process. A database and persistence across application restarts are **not required**.
- Include focused automated tests for the core domain behavior.
- Include a short README explaining the project structure and how to run the service and its tests locally.
- Do not implement features listed under *Out of scope*.

Delivery and startup contract

Submit a Git repository or zipped folder containing a `Dockerfile` at its root. The application will be started with:

```
1 docker build -t cleaning-robot .
2 docker run --rm -p 8000:8000 cleaning-robot
3
```

Within 10 seconds, the service must:

- be reachable at `http://localhost:8000`;
- expose FastAPI documentation at `/docs` and its schema at `/openapi.json`;
- respond to the health check defined below.

All identifiers are server-generated non-empty strings. UUIDs are suitable.

Domain model

Coordinates

The map is a rectangular grid using zero-based coordinates:

- `(0, 0)` is the top-left tile;
- `x` increases toward the east (right);
- `y` increases toward the south (down).

Movement deltas are:

Direction	Change
north	<code>(x, y - 1)</code>
east	<code>(x + 1, y)</code>
south	<code>(x, y + 1)</code>
west	<code>(x - 1, y)</code>

A tile is either walkable or non-walkable. Moving outside the map is a collision, just like moving into a non-walkable tile.

Cleanliness

Every walkable tile has a dirty or clean state. Loading a map establishes its initial cleanliness. Cleaning can change this state, and the state remains in memory across cleaning sessions until another map is loaded.

The service supports two robot models:

- `basic` : performs and reports a cleaning operation on every walkable tile it visits, even when that tile is already clean;
- `premium` : performs and reports a cleaning operation only when the visited tile is currently dirty. It skips a clean tile.

Both models mark a tile clean whenever they perform a cleaning operation.

API contract

All bodies are JSON unless an endpoint explicitly specifies another format. Validation errors use HTTP `422` . The exact error body for ordinary validation failures is up to you.

GET /health

Return exactly:

- status: `200` ;
- body: `{"status":"ok"}` .

PUT /map

Load or replace the current map. Send a multipart form upload using the field name `file` .

The uploaded filename extension, compared case-insensitively, must be `.txt` or `.json` .

On success, return `200` with this summary:

```
1 {  
2   "rows": 3,  
3   "cols": 4,  
4   "walkable_tiles": 10  
5 }  
6
```

Loading a new map:

- replaces the previous map;
- resets all tile cleanliness according to the new file;
- does **not** erase cleaning-session history.

Return:

- `415` when the filename has an unsupported extension;
- `422` when the file contents do not describe a valid map.

TXT map format

A TXT map is a rectangular grid where:

- `o` means walkable and initially dirty;

- `x` means non-walkable;
- only lowercase `o` and `x` are valid tile characters;
- every row must be non-empty and have the same number of columns;
- the map must contain at least one row and one column;
- Unix (`\n`) and Windows (`\r\n`) line endings are accepted;
- one line ending after the last row is accepted;
- blank lines inside the grid are invalid.

Example:

```
1 | 0x00
2 | 00x0
3 | 0000
4 |
```

This map has three rows, four columns, and ten walkable tiles.

JSON map format

A JSON map contains positive `rows` and `cols` values and exactly one tile for every coordinate in the rectangle:

```
1 | {
2 |   "rows": 2,
3 |   "cols": 3,
4 |   "tiles": [
5 |     {"x": 0, "y": 0, "walkable": true, "dirty": true},
6 |     {"x": 1, "y": 0, "walkable": true, "dirty": false},
7 |     {"x": 2, "y": 0, "walkable": false, "dirty": false},
8 |     {"x": 0, "y": 1, "walkable": true},
9 |     {"x": 1, "y": 1, "walkable": true},
10 |    {"x": 2, "y": 1, "walkable": true}
11 |   ]
12 | }
13 |
```

Rules:

- `rows` and `cols` are positive JSON integers;
- `x` and `y` are integers within the declared bounds;
- each coordinate appears exactly once—missing and duplicate coordinates are invalid;
- `walkable` is a boolean, and `dirty` is an optional boolean; rejecting other JSON types coerced to a boolean (for example the string `"true"`) is not required;
- a walkable tile whose `dirty` field is omitted starts dirty;
- a non-walkable tile must have `dirty` omitted or set to `false` ; setting it to `true` is invalid.

POST /clean

Run one cleaning session on the current map.

Request:

```
1 {  
2   "start": {"x": 0, "y": 0},  
3   "robot_model": "basic",  
4   "actions": [  
5     {"direction": "east", "steps": 2},  
6     {"direction": "south", "steps": 1}  
7   ]  
8 }  
9
```

Rules:

- `start`, `robot_model`, and `actions` are required, even when `actions` is empty;
- `robot_model` is exactly `basic` or `premium`;
- `direction` is exactly one of `north`, `east`, `south`, or `west` in lowercase;
- `steps` is a positive integer;
- coordinate values are integers;
- `actions` may be empty;
- the start coordinates must identify a walkable tile in the current map;
- the starting tile is visited and processed before the first action;
- each action is executed one step at a time;
- `submitted_actions` is the number of action objects in the request;
- `successful_steps` counts successful movements and does not include processing the starting tile;
- `cleaned_tiles` is ordered by cleaning-operation time;
- repeated coordinates therefore appear more than once for a basic robot, but not for a premium robot unless the tile becomes dirty again after another map upload.

Return:

- `200` with a completed session report when all actions finish;
- `409` when no map has been loaded;
- `422` for a malformed request or a start coordinate that is outside the map or non-walkable;
- `409` with an error session report when a movement collides with an obstacle or map boundary.

Completed report

```
1 {  
2   "id": "d911422c-18a1-423b-8284-d8c70f769489",  
3   "started_at": "2026-07-11T09:30:00Z",  
4   "finished_at": "2026-07-11T09:30:00.012Z",  
5   "state": "completed",  
6   "robot_model": "basic",  
7 }
```

```

7   "submitted_actions": 2,
8   "successful_steps": 3,
9   "cleaned_tiles": [
10    {"x": 0, "y": 0},
11    {"x": 1, "y": 0},
12    {"x": 2, "y": 0},
13    {"x": 2, "y": 1}
14  ],
15  "final_position": {"x": 2, "y": 1},
16  "duration_ms": 12,
17  "error": null
18 }
19

```

`started_at` and `finished_at` are RFC 3339 timestamps in UTC. `duration_ms` is a non-negative integer.

Collision behavior and report

When the next step would enter a non-walkable tile or leave the map:

1. stop before entering the invalid coordinate;
2. do not execute any later step or action;
3. preserve cleaning operations already performed;
4. leave `final_position` at the last valid coordinate;
5. add the session to history with state `error`;
6. return HTTP `409` with a report using the same fields as a completed report.

For an error report:

- `state` is `error`;
- `error.code` is exactly `collision`;
- `error.position` is the coordinate the robot attempted to enter, even when it is outside the map;
- `error.message` is a non-empty human-readable string.

Example error field:

```

1  {
2    "code": "collision",
3    "message": "The robot cannot enter a non-walkable tile.",
4    "position": {"x": 3, "y": 0}
5  }
6

```

Premium-model example

Given this one-row map:

```

1  {
2    "rows": 1,
3    "cols": 2,
4    "tiles": [

```

```

5     {"x": 0, "y": 0, "walkable": true, "dirty": true},
6     {"x": 1, "y": 0, "walkable": true, "dirty": false}
7   ]
8 }
9

```

A premium session starting at `(0, 0)` and moving east once returns:

```

1 | "cleaned_tiles": [{"x": 0, "y": 0}]
2 |

```

Running the same premium session again, without reloading the map, returns:

```

1 | "cleaned_tiles": []
2 |

```

GET /history

Download cleaning-session history as RFC 4180-compatible CSV.

Return `200` with a `text/csv` media type and this exact header order:

```
id,started_at,state,robot_model,submitted_actions,successful_steps,cleaned_tiles,duration_ms
```

Rules:

- include both completed and error sessions;
- return sessions in creation order, oldest first, including when timestamps have equal precision;
- `id` and `started_at` exactly match the corresponding JSON report values;
- `cleaned_tiles` is the number of entries in the report's `cleaned_tiles` array;
- timestamps and numeric values use the same semantics as the JSON report;
- return the header followed by no data rows when history is empty;
- retain history when a new map is loaded;
- persistence across application restarts is not required.

Example:

```
id,started_at,state,robot_model,submitted_actions,successful_steps,cleaned_tiles,duration_ms
d911422c-18a1-423b-8284-d8c70f769489,2026-07-11T09:30:00Z,completed,basic,2,3,4,12
```

Out of scope

Do not implement:

- authentication or authorization;
- concurrent cleaning jobs or synchronization guarantees;

- persistence across application restarts;
- map versioning or map history;
- a frontend;
- cloud deployment or production infrastructure;
- additional robot models, directions, or tile types.

Evaluation priorities

In order of importance:

1. **Correctness** — behavior matches the documented API contract, including its edge cases.
2. **Professional engineering practices** — domain modeling and separation of concerns, readable and appropriately sized abstractions, meaningful tests of important domain behavior, idiomatic FastAPI usage, and documentation clear enough for someone else to run and extend the service.

No single element of the second priority (for example, having *a* test suite or *a* README) is sufficient on its own — what matters is whether the engineering judgment behind the whole submission would hold up in a real codebase.

Hidden tests use inputs and combinations not shown in the examples, but they do not test undocumented behavior.